

GardenOS — Dossier de défense orale

Projet SGBD — Electron + Prisma + Angular + SQLite Application de gestion d'un jardin potager.

Ce document a deux usages :

1. **Récapitulatif technique ultra-précis** de tout le code (ce qui existe, où, et pourquoi).
2. **Préparation à l'oral** : flux complets, traduction « bouton → SQL », questions/réponses anticipées, et défense honnête des points faibles.

Rappel barème : Fonctionnalité 10 % · Architecture/qualité 10 % · Modélisation 10 % · **Compréhension orale 70 %**. Tout se joue sur votre capacité à *expliquer vos choix* et à faire le *lien avec le SQL*.

Table des matières

1. Synthèse express (à dire en 2 minutes)
2. La stack et pourquoi ces choix
3. Architecture Electron : 3 processus + cycle de vie d'un appel
4. Le flux complet illustré (bouton → écran)
5. Modélisation de la base : les 12 modèles
6. Prisma → SQL : table de correspondance
7. « Derrière chaque bouton » : 8 flux concrets avec leur SQL
8. La couche Angular, concept par concept (avec emplacements)
9. Checklist de conformité aux consignes
10. Questions d'oral probables + réponses modèles
11. Points faibles assumés et comment les défendre
12. Annexe : glossaire & commandes
13. Défense des points faibles — réponses d'oral prêtes
14. Questions réellement posées à l'oral — réponses + exemples

1. Synthèse express (à dire en 2 minutes)

« GardenOS est une application de bureau **Electron** pour gérer un potager : un catalogue de **plantes**, un **jardin** découpé en **parcelles**, des **cultures** (une plante semée dans une parcelle), leurs **récoltes**, un **stock** de graines/outils, et un **journal de bord**. L'architecture suit le modèle Electron en **trois processus** : le **main** (Node.js) détient l'accès à la base via **Prisma** ; le **preload** expose une API sécurisée au front ; le **renderer** est une application **Angular** qui n'accède jamais directement à la base, seulement via des canaux **IPC**. La base est en **SQLite local** (aucun cloud), modélisée avec Prisma : **12 modèles**, des relations **1:N**, une table de jonction **N:M** (`CultureTag`), un **enum** (`Exposition`), des comportements **ON DELETE** explicites, et des `include` (jointures). Côté Angular : composants **standalone**, **signals**, **computed**, **effect**, formulaires **réactifs**, **routing**, et un découpage **service / composant** strict. »

Si on ne retient qu'une phrase : **le renderer demande → le preload transmet → le main exécute Prisma → SQLite répond → ça remonte en sens inverse.**

2. La stack et pourquoi ces choix

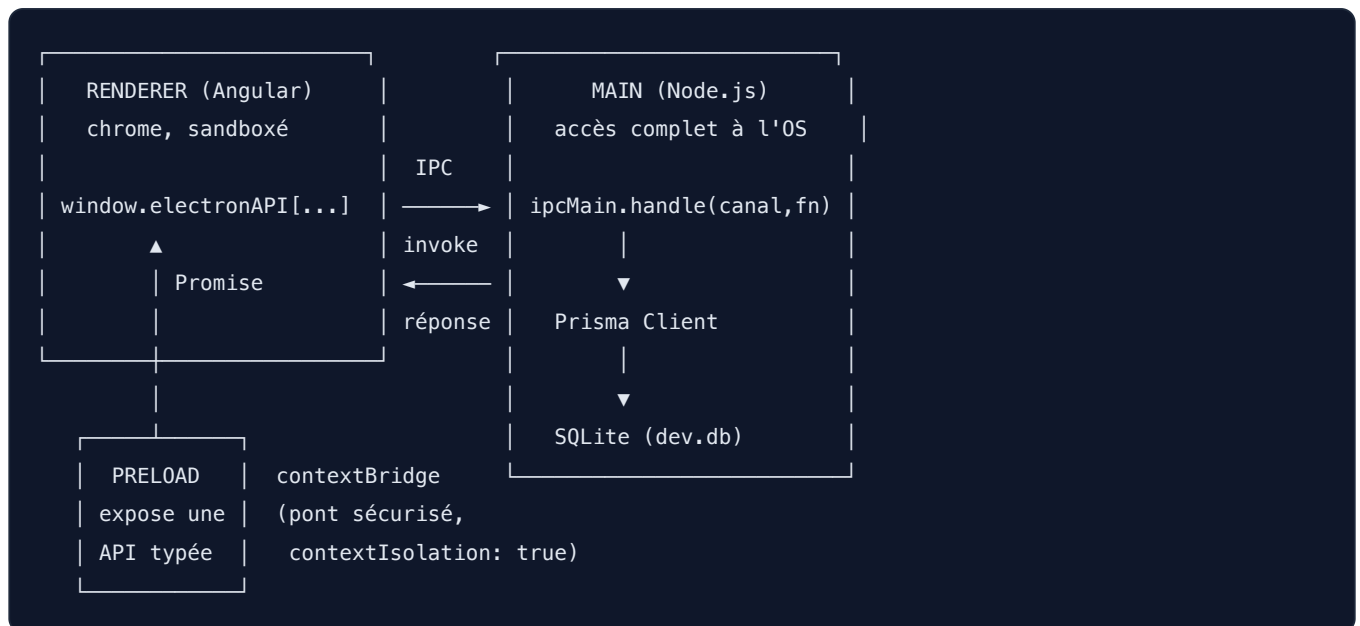
Couche	Techno	Rôle
Application de bureau	Electron 41	Fenêtre native multi-plateforme
Backend (main)	Node.js + Prisma 7	Logique d'accès aux données
Base de données	SQLite (fichier local)	Persistance, zéro dépendance cloud
ORM	Prisma + adaptateur <code>better-sqlite3</code>	Mapping objet↔relationnel, migrations
Frontend (renderer)	Angular 21	Interface utilisateur réactive
Bundler main/preload	Vite (via electron-forge)	Compilation TypeScript
Empaquetage	Electron Forge	<code>package</code> / <code>make</code>

Points de vocabulaire à maîtriser :

- **Prisma 7** utilise le nouveau generator `prisma-client` (sortie dans `generated/prisma`, voir `prisma/schema/_config.prisma`) et un **driver adapter** (`@prisma/adapters-better-sqlite3`) plutôt que l'ancien moteur binaire. C'est pour ça qu'on voit un compilateur de requêtes WASM dans `.vite/build`.
- **better-sqlite3** est un **module natif** : il est compilé en C++ pour un ABI Node précis. Electron embarque sa propre version de Node/V8 (ABI différent), d'où l'étape `npm run electron-rebuild -f -w better-sqlite3` du README. Si on saute cette étape : erreur « `NODE_MODULE_VERSION` mismatch ».
- **Vite externalise** `better-sqlite3` (`vite.main.config.ts`, `external: ['better-sqlite3', /\.node$/]`) pour ne pas tenter de bundler un binaire natif.

3. Architecture Electron : 3 processus + cycle de vie d'un appel

3.1 Les trois processus



- **Main** — `src/main/main.ts` : crée la `BrowserWindow` (lignes 21-38), enregistre tous les handlers IPC au démarrage (`app.on('ready')` , lignes 40-47), et **ferme proprement Prisma** à la sortie (`app.on('before-quit')` → `disconnectDb()` , lignes 55-57). En dev il charge `http://localhost:4200` (serveur Angular) ; en prod il charge le HTML Angular compilé.
- **Preload** — `src/preload/preload.ts` : un seul `contextBridge.exposeInMainWorld('electronAPI', { ... })` . Chaque clé est un canal qui appelle `ipcRenderer.invoke('canal', data)` . C'est la **seule** surface exposée au renderer.
- **Renderer** — `renderer/` : Angular. Il ne connaît que `window.electronAPI` , typé par `renderer/src/electron.d.ts` .

3.2 Sécurité (à connaître absolument)

Dans `main.ts` , la fenêtre est créée avec :

```
webPreferences: {
  preload: path.join(__dirname, 'preload.js'),
  contextIsolation: true, // le renderer et le preload ont des contextes JS isolés
  nodeIntegration: false, // le renderer NE peut PAS faire require('fs'), etc.
}
```

Pourquoi c'est important : si `nodeIntegration` était à `true` , une faille XSS dans le renderer donnerait un accès Node complet (lecture disque, etc.). Ici, le renderer est sandboxé et ne peut faire QUE ce que le preload expose explicitement. C'est le principe du **moindre privilège**.

`forge.config.ts` renforce encore avec les **Fuses** (`OnlyLoadAppFromAsar` , désactivation de `RunAsNode` , etc.) et `asar: true` .

3.3 Le contrat partagé (`src/shared/ipc/`)

Le dossier `src/shared/ipc/` contient les **interfaces et DTOs** importés à la fois par le main (handlers) et par le renderer (services + `electron.d.ts`). C'est le **contrat unique** : si je change un DTO, le compilateur me signale toutes les ruptures des deux côtés. Exemple : `CreateCultureDto` (`jardin.ipc.ts` , lignes 86-96).

- **DTO** = *Data Transfer Object* : la forme exacte des données qui transitent. Les **Create*Dto** n'ont pas d' **id** ; les **Update*Dto** ont **id** obligatoire et le reste optionnel (mise à jour partielle).

4. Le flux complet illustré (le trajet canonique)

Scénario : l'utilisateur clique « Supprimer » sur un article de stock.

1. **Template** `stock.component.html` (ligne 55) :

```
<button (click)="supprimer(item.id)">Supprimer</button>
```

1. **Composant** `stock.component.ts` (lignes 130-137) :

```
async supprimer(id: number) {
  try {
    await this.stockService.delete(id); // (A) appel service
    this.stocks.update(liste => liste.filter(s => s.id !== id)); // (D) MAJ optimiste du signal
  } catch (err) { console.error('[stock:supprimer]', err); }
}
```

1. **Service** `stock.service.ts` (ligne 39) :

```
delete(id: number): Promise<void> {
  return window.electronAPI['stocks:delete']({ id }); // (B) passe par l'API preload
}
```

1. **Preload** `preload.ts` (ligne 25) :

```
'stocks:delete': (data) => ipcRenderer.invoke('stocks:delete', data); // franchit la frontière de process
```

1. **Handler (main)** `stock.handlers.ts` (lignes 67-74) :

```
ipcMain.handle('stocks:delete', async (_event, { id }) => {
  try { await db.stockItem.delete({ where: { id } }); } // (C) Prisma → SQLite
  catch (err) { console.error('[stocks:delete]', err); throw err; }
});
```

1. **Prisma** traduit en SQL : `DELETE FROM "StockItem" WHERE "id" = ?;`

2. La promesse se résout, remonte main → preload → service → composant, et le **signal** `stocks` est mis à jour → Angular re-render automatiquement la grille.

Les 4 frontières à nommer à l'oral : (A) composant↔service, (B) service↔preload, (C) handler↔Prisma, (D) réactivité signal↔template. Le passage **renderer** → **main** est le seul franchissement de *processus* (sérialisation par structured clone).


```

model Culture {
  parcelleId Int
  parcelle  Parcelle @relation(fields: [parcelleId], references: [id], onDelete: Cascade)
}
model Parcelle {
  cultures Culture[] // côté "plusieurs"
}

```

- Le **côté qui porte la clé étrangère** (`parcelleId`) est le côté « N ». `parcelle.prisma` n'a qu'un tableau `Culture[]` (champ virtuel, **pas** une colonne).
- **Lien SQL** : la FK `Culture.parcelleId` référence `Parcelle.id`. C'est exactement un `FOREIGN KEY ("parcelleId") REFERENCES "Parcelle" ("id")`.

Autres 1:N du projet : `Plante` → `Culture`, `StatutCulture` → `Culture`, `TypePlante` → `Plante`, `TypeSol` → `Parcelle`, `CategorieStock` → `StockItem`, `Plante` → `StockItem`, `Culture` → `Recolte`, `Culture` → `Journal`.

5.4 Relation N:M (plusieurs-à-plusieurs) — table de jonction explicite

`tag.prisma` :

```

model Tag {
  id      Int      @id @default(autoincrement())
  libelle String    @unique
  cultures CultureTag[]
}
model CultureTag {
  cultureId Int
  culture   Culture @relation(fields: [cultureId], references: [id], onDelete: Cascade)
  tagId     Int
  tag       Tag     @relation(fields: [tagId], references: [id], onDelete: Cascade)
  @@id([cultureId, tagId]) // PK composite = empêche les doublons (culture, tag)
}

```

- Une culture a plusieurs tags, un tag concerne plusieurs cultures → **N:M**. On le matérialise par une **table pivot** `CultureTag` portant deux FK.
- **Pourquoi une table de jonction explicite** (et pas la relation implicite `Tag[] <-> Culture[]` de Prisma) ? Parce que la consigne l'exige (« table pivot explicite avec `@@id` ») et que c'est extensible : on pourrait y ajouter une colonne (date d'ajout du tag, etc.).
- **Équivalent SQL** : c'est le schéma classique de décomposition d'un N:M en deux 1:N autour d'une table d'association.

5.5 ON DELETE : trois comportements, choisis exprès

Relation	onDelete	SQL	Raison
Culture → Plante	Cascade	ON DELETE CASCADE	Si je supprime une plante du catalogue, ses cultures n'ont plus de sens
Culture → Parcelle	Cascade	ON DELETE CASCADE	Supprimer une parcelle supprime ses cultures
Recolte → Culture	Cascade	ON DELETE CASCADE	Une récolte sans culture est orpheline
Journal → Culture	Cascade	ON DELETE CASCADE	Idem
CultureTag → Culture / Tag	Cascade	ON DELETE CASCADE	Nettoie les liens de jonction
Culture → StatutCulture	Restrict (défaut)	ON DELETE RESTRICT	Interdit de supprimer un statut encore utilisé
Plante → TypePlante	Restrict (défaut)	ON DELETE RESTRICT	Interdit de supprimer un type encore utilisé
StockItem → CategorieStock	Restrict (défaut)	ON DELETE RESTRICT	Idem
Parcelle → TypeSol (optionnel)	SetNull (défaut)	ON DELETE SET NULL	Le sol redevient « non défini »
StockItem → Plante (optionnel)	SetNull (défaut)	ON DELETE SET NULL	L'article perd juste son lien plante

À retenir : quand je n'écris pas `onDelete`, Prisma applique un défaut qui dépend de l'obligation du champ — **Restrict** si la FK est obligatoire, **SetNull** si elle est optionnelle (`Int?`). Je peux le **prouver** en montrant le SQL généré dans `prisma/migrations/20260518134503_init/migration.sql` (les `ON DELETE RESTRICT`, `SET NULL`, `CASCADE` y sont écrits noir sur blanc).

Sur SQLite, les cascades fonctionnent grâce à `PRAGMA foreign_keys = ON`. C'est le moteur SQLite qui propage la suppression, pas Prisma.

5.6 Enum

`parcelle.prisma` :

```
enum Exposition { PLEIN_SOLEIL MI_OMBRE OMBRE }
model Parcelle { exposition Exposition? }
```

- **Point clé à connaître :** SQLite **n'a pas de type enum natif**. Prisma stocke l'enum comme une colonne `TEXT` (`"exposition" TEXT`). C'est *exactement* pour ça que la migration `20260531185639_add_exposition_enum` est **vide** (`-- This is an empty migration`) : la colonne était déjà du `TEXT`, transformer un `String?` en `enum Exposition?` ne change rien au schéma SQL, seulement la **validation côté Prisma/TypeScript**.
- Côté front, l'enum est ré-exprimé en union de types : `type Exposition = 'PLEIN_SOLEIL' | 'MI_OMBRE' | 'OMBRE'` (`jardin.ipc.ts`, ligne 2).

5.7 Champs optionnels (nullable)

Beaucoup : `Plante.nomLatin String?` , `description String?` , `Culture.dateSemisReelle DateTime?` , `StockItem.seuilAlerte Float?` , etc. En SQL, l'absence de `NOT NULL` . La distinction *prévu* vs *réel* sur les dates de culture (`dateSemisPrevue` obligatoire / `dateSemisReelle?` optionnelle) est un vrai choix métier : on planifie d'abord, on constate ensuite.

5.8 Historique des migrations (sachez le raconter)

7 migrations dans `prisma/migrations/` , qui **racontent l'évolution du modèle** :

1. `...082607_init` : tables `Garden` / `Plant` — **vestiges du template** todos-app de départ.
2. `...134503_init` : on **supprime** `Garden/Plant` et on crée le vrai schéma FR. À ce stade il y avait *en plus* `Alerte` , `Association` , `TypeAlerte` , `TypeAssociation` .
3. `...typeplante_optional` : `typePlanteId` devient nullable.
4. `...add_parcelle_pos` : ajout de `posX` / `posY` (grille du jardin) + `typePlanteId` redevient obligatoire.
5. `...remove_journal_humeur` : suppression de la colonne `humeur` du journal.
6. `...remove_alerte_association` : suppression des 4 tables `Alerte/Association` (simplification du périmètre).
7. `...add_exposition_enum` : **vide** (enum = TEXT, cf. 5.6).

La chaîne est **cohérente et rejouable** : `npx prisma migrate deploy` reconstruit la base depuis zéro. Les tables `Garden/Plant` n'existent plus dans la base finale (supprimées en migration 2). Sur SQLite, les modifs de colonnes passent par la technique « **table neuve + copie + rename** » (`RedefineTables`), visible dans les migrations 3-5 — c'est parce que SQLite ne sait pas faire `ALTER COLUMN` .

6. Prisma → SQL : table de correspondance

Nuance importante à dire à l'oral : un `include` Prisma n'émet pas forcément un `JOIN`. Par défaut, Prisma exécute **plusieurs `SELECT`** (un par relation) qu'il **assemble en mémoire**. On peut forcer un vrai JOIN SQL avec `relationLoadStrategy: "join"`. Je donne ci-dessous l'**équivalent logique** (ce que l'examineur veut voir) ; le résultat est identique, c'est la stratégie d'exécution qui diffère.

Appel Prisma	SQL (équivalent logique)
<code>db.plante.findMany({ orderBy: { nom: 'asc' } })</code>	<code>SELECT * FROM "Plante" ORDER BY "nom" ASC;</code>
<code>db.plante.count()</code>	<code>SELECT COUNT(*) FROM "Plante";</code>
<code>db.plante.findUnique({ where: { id } })</code>	<code>SELECT * FROM "Plante" WHERE "id" = ? LIMIT 1;</code>
<code>findMany({ include: { typePlante: true } })</code>	<code>SELECT p.*, t.* FROM "Plante" p JOIN "TypePlante" t ON t."id" = p."typePlanteId";</code>
<code>db.plante.create({ data })</code>	<code>INSERT INTO "Plante" (...) VALUES (...) RETURNING *;</code>
<code>db.plante.update({ where:{id}, data })</code>	<code>UPDATE "Plante" SET ... WHERE "id" = ?;</code>
<code>db.plante.delete({ where:{id} })</code>	<code>DELETE FROM "Plante" WHERE "id" = ?;</code>
<code>db.journal.findMany({ where:{cultureId}, orderBy:{date:'desc'} })</code>	<code>SELECT * FROM "Journal" WHERE "cultureId" = ? ORDER BY "date" DESC;</code>
<code>db.tag.connectOrCreate({ where:{libelle}, create:{libelle} })</code>	<code>SELECT id FROM "Tag" WHERE "libelle"=?; puis INSERT si absent</code>

Le **RETURNING *** : Prisma récupère immédiatement la ligne créée/modifiée (avec l'`id` auto-incrémenté) — c'est ce qui permet aux handlers de renvoyer l'objet complet au front sans 2e requête.

7. « Derrière chaque bouton » : 8 flux concrets avec leur SQL

Flux 1 — Ouvrir la page « Plantes » (lecture + jointure)

Bouton/route : `/plantes` . `PlantesComponent.ngOnInit` appelle `planteService.getAll()` → canal `plantes:getAll` → `plante.handlers.ts:24` :

```
db.plante.findMany({ include: { typePlante: true }, orderBy: { nom: 'asc' } });
```

SQL :

```
SELECT p.*, t."libelle"
FROM "Plante" p
JOIN "TypePlante" t ON t."id" = p."typePlanteId"
ORDER BY p."nom" ASC;
```

Le résultat (typé `Plante[]` avec `typePlante` inclus) remonte, est stocké dans le signal `plantes`, et le `computed` `plantesFiltrees` l'affiche via `@for`.

Flux 2 — Dashboard : le comptage (agrégat) + agrégation applicative

`dashboard.component.ts:38-51` charge **3 sources en parallèle** avec `Promise.all` : `getParcelles()`, `stockService.getAll()`, et `planteService.count()`.

- `count()` → `plante.handlers.ts:19` → `db.plante.count()` → `SELECT COUNT(*) FROM "Plante";` → affiché dans la carte « Plantes dans le catalogue » (`dashboard.component.html:9`). **C'est l'agrégat exigé par la consigne, visible dans l'UI.**
- Les autres stats (`cultures().length`, `totalRecoltes`, `alertes`) sont calculées **côté client** par des `computed` à partir des parcelles déjà chargées (avec leurs cultures/récoltes incluses). À l'oral : je sais distinguer **agrégat SQL** (`COUNT`) et **agrégation applicative** (réduction JS sur des données déjà jointes).

Flux 3 — Créer une culture avec des tags (écriture imbriquée + N:M + transaction)

Bouton « Sauvegarder » de la modale culture → `jardin.component.ts:260` `createCulture(...)` → `jardin.handlers.ts:123-141` :

```
db.culture.create({
  data: {
    ...data,
    dateSemisPrevue: toDate(dateSemisPrevue)!, // ISO string → Date
    tags: { create: tags.map(libelle => ({
      tag: { connectOrCreate: { where: { libelle }, create: { libelle } } }
    })) },
  },
  include: includeCulture,
});
```

SQL (dans une transaction) :

```
BEGIN;
INSERT INTO "Culture" ("dateSemisPrevue","dateRecoltePrevue","planteId","parcelleId","statutId",...)
VALUES (?, ?, ?, ?, ?, ...) RETURNING *;           -- id de la nouvelle culture
-- pour chaque tag : connectOrCreate
SELECT "id" FROM "Tag" WHERE "libelle" = ?;         -- existe ?
INSERT INTO "Tag" ("libelle") VALUES (?) RETURNING *; -- si absent
INSERT INTO "CultureTag" ("cultureId","tagId") VALUES (?, ?);
-- include : relit la culture + plante + statut + tags + recoltes
COMMIT;
```

Trois points d'or pour l'oral :

- `connectOrCreate` = « relie au tag existant, sinon crée-le » → évite les doublons grâce à `Tag.libelle @unique`.
- Tout est dans une transaction** : si une étape échoue, rien n'est écrit (atomicité — le « A » d'ACID).
- Conversion de dates** : les DTOs traversent l'IPC en **chaînes ISO** (un `<input type="date">` renvoie `"2026-03-15"`). Le handler les reconvertit en `Date` via `toDate()` avant Prisma (`jardin.handlers.ts:35-38`).

Flux 4 — Modifier les tags d'une culture (remplacement complet)

`jardin.handlers.ts:158-163` :

```
tags: { deleteMany: {}, create: [...] }
```

SQL :

```
DELETE FROM "CultureTag" WHERE "cultureId" = ?; -- on vide les liens existants
-- puis on recrée les liens (connectOrCreate comme au flux 3)
```

Choix assumé : **on remplace toute la liste** plutôt que de calculer un diff ajout/retrait. Plus simple, suffisant ici, et la PK composite empêche les doublons.

Flux 5 — Drag & drop d'une parcelle sur la grille (update ciblé)

jardin.component.ts:85-97 onDrop → updateParcelle({ id, posX, posY }) → jardin.handlers.ts:98-106 . **SQL** : UPDATE "Parcelle" SET "posX" = ?, "posY" = ? WHERE "id" = ?; La grille 4x4 est un `computed` (`grille`, lignes 60-69) qui place chaque parcelle selon `posX/posY` ; après l'update on met à jour le signal `parcelles` et le `computed` recalcule la grille. **Réactivité de bout en bout sans recharger la page.**

Flux 6 — Supprimer une parcelle (cascade)

parcelles:delete → db.parcelle.delete({ where: { id } }) . **SQL** : DELETE FROM "Parcelle" WHERE "id" = ?; **Effet en chaîne** (grâce aux `ON DELETE CASCADE`) : SQLite supprime les `Culture` de la parcelle, ce qui cascade vers leurs `Recolte`, `Journal` et lignes `CultureTag`. Une seule instruction, suppression propre de tout le sous-arbre.

Flux 7 — Supprimer un référentiel encore utilisé (RESTRICT + gestion d'erreur visible)

Page Paramètres, bouton X sur « Légume » → categoriesStock:delete (ou typePlantes, etc.). **SQL** : DELETE FROM "CategorieStock" WHERE "id" = ?; Si des `StockItem` y font référence, la contrainte `ON DELETE RESTRICT` fait **échouer** la requête (SQLite renvoie une erreur de contrainte de clé étrangère, Prisma lève une erreur). Le composant **l'attrape et l'affiche** (`parametres.component.ts:75-88`) :

```
const isFk = err?.message?.includes('Foreign key') || err?.message?.includes('foreign key');
s.erreur = isFk ? `Impossible de supprimer : cette valeur est encore utilisée.` : `Erreur lors de la suppression.`;
```

C'est mon meilleur exemple de gestion d'erreur métier remontée jusqu'à l'utilisateur. (Commit `9c0eb20` "Show deletion error for referenced items".)

Flux 8 — Import Wikipedia (appel réseau externe, hors IPC)

Modale « Ajouter une plante » → bouton « Rechercher » → plantes.component.ts:79 rechercherWikipedia() → wikipedia.service.ts .

- `fetch("https://fr.wikipedia.org/api/rest_v1/page/summary/<nom>")` , puis si besoin Wikidata pour le nom latin (propriété **P225**, *taxon name*).
- Le résultat **pré-remplit le formulaire réactif** (`form.patchValue({...})`).
- **Aucune base touchée** ici : c'est juste de l'aide à la saisie. La plante n'est persistée qu'au clic « Sauvegarder » (flux create classique).
- **Honnêteté** : cet appel part **directement du renderer** (et non via un handler IPC du main). C'est une entorse à la philosophie Electron, déjà notée dans mon README. Voir §11.

8. La couche Angular, concept par concept (avec emplacements)

8.1 Composants standalone (≥ 3 exigés → 9 présents)

Tous les composants ont `standalone: true` : `App` (`app.ts`), `NavbarComponent`, `DashboardComponent`, `JardinComponent`, `PlantesComponent`, `PlanteCardComponent`, `StockComponent`, `JournalComponent`, `ParametresComponent`. **Plus de composants NgModule du tout** — c'est l'approche moderne Angular : chaque composant déclare ses propres `imports`.

8.2 TypeScript : interfaces

- `Plante`, `TypePlante`, `CreatePlanteDto`, `UpdatePlanteDto` (`plante.service.ts`), `JournalEntry`, `StockItem`, `CultureFull`, `ParcelleFull`, etc.
- Les types « source de vérité » vivent dans `src/shared/ipc/` et sont **réexportés** par les services (`export type { ... }`) pour que les composants ne dépendent que du service.

8.3 Signals (état local réactif)

Partout : `signal<StockItem[]>([ ])`, `recherche = signal('')`, `modalOuvert = signal(false)` ... Mise à jour via `.set(...)` et `.update(liste => ...)` (ex. `stock.component.ts:110` `this.stocks.update(liste => liste.map(...))`). Lecture dans le template en **appelant** le signal : `{{ stocks() }}`.

8.4 Computed (au moins 1 → une douzaine)

- `dashboard` : `cultures`, `totalRecoltes`, `alertes`, `parStatut`, `prochainsSevis`, `prochainesRecoltes`.
- `stock` : `stocksFiltres`. `plantes` : `plantesFiltrees`. `jardin` : `grille`. `journal` : `cultures`, `entreesFiltrees`.
- Pédagogie** : un `computed` se **recalcule automatiquement** quand un des signals qu'il lit change, et **met en cache** sinon. Ex. `stocksFiltres` dépend de `stocks()`, `recherche()` et `filtreCategorie()` ; taper dans la recherche refiltre la liste sans aucun appel base.

8.5 effect() (bonus — présent 3×)

- `dashboard.component.ts:31-34` : warn console quand des stocks passent sous le seuil.
- `stock.component.ts:47-51` : warn quand des articles sont épuisés.
- `journal.component.ts:61-64` : **remet la recherche à zéro** dès qu'on change le filtre de culture (effet de bord réactif).
- Pédagogie : `effect` sert aux **effets de bord** (log, synchro), pas à produire une valeur (ça, c'est `computed`).

8.6 @for / @if / @empty

Nouvelle syntaxe de control flow partout. Ex. `stock.component.html` : `@for (item of stocksFiltres(); track item.id) { ... } @empty { ... }`, `@if (enAlerte(item)) { ... }`. Le `track` (sur `item.id`) permet à Angular de réutiliser les nœuds DOM au lieu de tout recréer — performance + état préservé.

8.7 Service + injection de dépendances (DI) + Singleton

- Tous les services : `@Injectable({ providedIn: 'root' })` → **une seule instance** partagée dans toute l'app (singleton). Si deux composants injectent `JardinService`, c'est le **même** objet.
- Injection par la fonction `inject()` : `private jardinService = inject(JardinService)` (style moderne, pas de constructeur).
- Séparation des responsabilités** : les services ne font *que* l'accès données (appels IPC) ; les composants gèrent l'état d'UI et orchestrent. C'est le critère « logique métier dans les services » des consignes.

8.8 input() / output() (composant parent ↔ enfant)

`PlanteCardComponent` (composant « muet » de présentation) :

```
plante = input.required<Plante>(); // données parent → enfant
editer = output<Plante>();          // événements enfant → parent
supprimer = output<number>();
```

Le parent `plantes.component.html:21–25` :

```
<app-plante-card [plante]="plante" (editer)="ouvrirEdition($event)" (supprimer)="supprimer($event)" />
```

C'est l'exemple canonique **smart/dumb component** : `PlantesComponent` détient la logique, `PlanteCardComponent` affiche et **remonte** les intentions via `output`.

8.9 Formulaires : les trois approches présentes

Approche	Où	Comment
Réactif (<code>ReactiveFormsModule</code>)	<code>plantes.component.ts:26–38</code>	<code>fb.group({ nom: ['', Validators.required], ... })</code> , lié au template par <code>[formGroup]</code> + <code>formControlName</code> , bouton désactivé si <code>form.invalid</code>
Template (<code>FormsModule</code> , <code>[(ngModel)]</code>)	<code>parametres.component.html:13</code>	<code>[(ngModel)]="s.libelle"</code> (two-way binding)
Template signal-friendly (<code>FormsModule</code>)	<code>journal.component.html</code>	<code>[ngModel]="formContenu()"</code> + <code>(ngModelChange)="formContenu.set(\$event)"</code>
Signal + binding natif	<code>stock</code> , <code>jardin</code>	<code>[value]</code> + <code>(input)="formStock.update(...)"</code> sans module de forms

La consigne demande « au moins un formulaire avec `ReactiveFormsModule` » → c'est `PlantesComponent`, avec validations. Les autres montrent que je maîtrise aussi `ngModel` et l'approche full-signals.

8.10 Routing + RouterLink

- `app.routes.ts` : **7 routes** (`''` redirige vers `dashboard`, puis `dashboard/jardin/plantes/stock/journal/parametres`). Bien plus que les 2 exigées.
- `app.config.ts` : `provideRouter(routes)`.
- `app.ts` : `<router-outlet />` dans le template.
- `navbar.component.html` : `routerLink="/jardin"` + `routerLinkActive="active"` (surligne l'onglet courant). `NavbarComponent` importe `RouterLink`, `RouterLinkActive`.

9. Checklist de conformité aux consignes

9.1 Angular (annexe 5.1)

Notion	Statut	Preuve
≥ 3 composants standalone	✓ (9)	tous les <code>@Component({ standalone: true })</code>
Interfaces TypeScript	✓	<code>shared/ipc/*</code> , <code>plante.service.ts</code>
Signals	✓	<code>signal(...)</code> dans tous les composants
Computed	✓ (≥12)	<code>dashboard</code> , <code>stock</code> , <code>plantes</code> , <code>jardin</code> , <code>journal</code>
@for / @if	✓	tous les templates (+ <code>@empty</code>)
Service + DI	✓	6 services, <code>inject(...)</code>
Singleton	✓	<code>providedIn: 'root'</code>
input()	✓	<code>plante-card</code> <code>input.required<Plante>()</code>
output()	✓	<code>plante-card</code> <code>output<Plante>()</code> / <code>output<number>()</code>
Formulaire réactif	✓	<code>plantes.component.ts</code> (<code>fb.group</code> + <code>Validators</code>)
Routage ≥ 2 routes + outlet	✓ (7)	<code>app.routes.ts</code> , <code>provideRouter</code> , <code><router-outlet></code>
RouterLink	✓	<code>navbar</code> (<code>routerLink</code> + <code>routerLinkActive</code>)
effect() (bonus)	✓	<code>dashboard</code> , <code>stock</code> , <code>journal</code>

9.2 Prisma / SQL (annexe 5.2)

Notion	Statut	Preuve
≥ 7 modèles	✓ (12)	<code>prisma/schema/*.prisma</code>
Clé primaire	✓	<code>id Int @id @default(autoincrement())</code> partout
Relation 1:N + FK + onDelete	✓	<code>Parcelle</code> → <code>Culture</code> , etc.
Table de jonction N:M	✓	<code>CultureTag</code> <code>@@id([cultureId, tagId])</code>
ON DELETE	✓	Cascade / Restrict / SetNull (cf. §5.5)
JOIN / include	✓	<code>include</code> dans tous les handlers
Agrégat / comptage dans l'UI	✓	<code>plante.count()</code> → carte Dashboard
CRUD complet	✓	Plante, Culture, StockItem, Journal, Parcelle, refs
Champs optionnels	✓	nombreux <code>String?</code> / <code>Float?</code> / <code>DateTime?</code>
Enum (bonus)	✓	<code>Exposition</code>

9.3 Architecture Electron (annexe 5.3) & qualité (2.3)

Notion	Statut	Preuve / nuance
Main process	✓	<code>main.ts</code> crée la <code>BrowserWindow</code> , charge l'app
Preload (contextBridge)	✓	<code>preload.ts</code> expose <code>electronAPI</code>
Renderer via preload uniquement	⚠	OUI sauf <code>wikipedia.service.ts</code> (fetch direct, cf. §11)
IPC <code>handle</code> / <code>invoke</code>	✓	handlers + preload
<code>npm run start</code> unique	✓	<code>concurrently</code> Angular + Electron
SQLite local, pas de cloud	✓	<code>provider = "sqlite"</code> , <code>dev.db</code> (Wikipedia = enrichissement optionnel)
TypeScript strict	⚠	renderer strict ; <code>tsconfig.json</code> racine seulement <code>noImplicitAny</code> (cf. §11)
Modularité (1 responsabilité, logique en service)	✓	services = IPC, composants = UI
Nommage explicite	✓	<code>parcelleSelectionnee</code> , <code>stocksFiltres</code> ... (pas de <code>data</code> / <code>temp</code>)
Gestion d'erreurs (try/catch IPC)	⚠	présent partout sauf <code>plante.handlers.ts</code> (cf. §11)
Documentation	✓	README complet + JSDoc sur quasi toutes les fonctions
Schéma draw.io	✓	<code>garden0s_Bd.drawio</code> (+ PDF)
Script de seed	✓	<code>prisma/seed.ts</code> (référentiels, plantes, parcelles, cultures, récoltes, journal, stock)

10. Questions d'oral probables + réponses modèles

Architecture & Electron

Q : Pourquoi 3 processus ? Pourquoi ne pas mettre Prisma dans Angular ? R : Prisma utilise des API Node.js (système de fichiers, module natif `better-sqlite3`) indisponibles dans un navigateur. Le renderer est un environnement Chromium sandboxé. On place donc tout l'accès données dans le **main** (vrai Node) et on communique par **IPC**. Avantage sécurité : le renderer ne peut pas toucher au disque directement.

Q : Que fait exactement le preload ? Pourquoi pas exposer `ipcRenderer` directement ? R : Le preload est le **pont contrôlé**. Avec `contextIsolation: true`, on ne peut passer du main au renderer que via `contextBridge`. Exposer `ipcRenderer` brut laisserait le renderer appeler *n'importe quel* canal ; à la place j'expose une **liste blanche** de méthodes précises (`'stocks:delete'`, etc.). Surface d'attaque minimale.

Q : Comment les données traversent-elles l'IPC ? Et les dates ? R : `ipcRenderer.invoke` sérialise les arguments avec l'algorithme de **clonage structuré**. Les objets simples passent tels quels. Pour les dates, je les fais transiter en **chaînes ISO** (le `<input type=date>` produit déjà une string) et le handler les reconvertit en `Date` avec `toDate()` avant de les donner à Prisma — c'est explicite et sans ambiguïté.

Q : `npm run start` fait quoi ? R : Via `concurrently`, il lance **deux** process : (1) le serveur de dev Angular sur `:4200`, (2) `wait-on` attend que `:4200` réponde puis démarre `electron-forge start`. En dev la fenêtre charge `localhost:4200` (hot reload) ; en prod elle charge le HTML buildé.

Prisma & SQL

Q : Montrez une relation 1:N et le SQL correspondant. R : `Culture.parcelleId + @relation(... onDelete: Cascade)`. En SQL : `FOREIGN KEY ("parcelleId") REFERENCES "Parcelle"("id") ON DELETE CASCADE`. Le côté « 1 » (`Parcelle`) n'a qu'un tableau virtuel `cultures Culture[]`, pas de colonne.

Q : Expliquez votre table de jonction. Pourquoi `@id([cultureId, tagId])` ? R : C'est la décomposition du N:M `Culture ↔ Tag`. La **PK composite** garantit qu'un même couple (culture, tag) ne peut exister qu'une fois — pas de doublon de lien. En SQL : `PRIMARY KEY ("cultureId", "tagId")` + deux FK.

Q : Que génère un `include` ? Est-ce un JOIN ? R : Logiquement, oui, c'est une jointure sur la FK. En pratique, par défaut Prisma exécute **plusieurs SELECT** et assemble côté client ; on peut forcer un vrai JOIN avec `relationLoadStrategy: "join"`. Le résultat est le même objet imbriqué.

Q : Où est l'agrégat ? R : `db.plante.count()` (`SELECT COUNT(*)`) affiché sur le Dashboard. Je distingue ça des stats calculées en JS par des `computed` sur des données déjà chargées.

Q : Pourquoi la migration d'enum est-elle vide ? R : SQLite n'a pas d'enum natif → Prisma le stocke en `TEXT`. La colonne `exposition` était déjà `TEXT`, donc passer de `String?` à `enum Exposition?` ne change pas le SQL, seulement la validation côté TypeScript/Prisma.

Q : Que se passe-t-il si je supprime un « Statut » utilisé ? Et une parcelle ? R : Statut → `RESTRICT` : la suppression **échoue**, l'erreur de FK est attrapée et affichée à l'utilisateur (Paramètres). Parcelle → `CASCADE` : SQLite supprime en chaîne les cultures, et donc leurs récoltes, entrées de journal et liens de tags.

Q : Vos suppressions imbriquées sont-elles atomiques ? R : Oui. Une `create / update` avec écritures imbriquées (tags) est encapsulée par Prisma dans une **transaction** : tout ou rien.

Angular

Q : Différence `signal` / `computed` / `effect` ? R : `signal` = source d'état mutable. `computed` = valeur **dérivée**, recalculée à la demande et mise en cache. `effect` = **effet de bord** déclenché à chaque changement des signaux lus (log, reset...). Règle : si ça produit une valeur affichée → `computed` ; si ça agit sur le monde extérieur → `effect`.

Q : Pourquoi `providedIn: 'root'` ? R : Ça enregistre le service comme **singleton** au niveau racine : une instance unique, tree-shakable, partagée par DI dans toute l'app.

Q : Comment parent et enfant communiquent (plantes / plante-card) ? R : `input.required<Plante>()` descend la donnée, `output<Plante>()` / `output<number>()` remontent les événements `edit` / `supprimer`. Le parent garde la logique, l'enfant ne fait qu'afficher et émettre.

Q : Pourquoi le `track` dans `@for` ? R : Il identifie chaque élément (par `id`) pour qu'Angular réutilise les nœuds DOM existants au lieu de tout reconstruire — meilleures perfs et préservation de l'état (focus, scroll).

Q : Mise à jour « optimiste » de la liste — pourquoi ne pas recharger depuis la base ? R : Après un create/update/delete, je mets à jour le **signal** localement (`.update(...)`) avec l'objet renvoyé par le handler. Évite un aller-retour réseau et garde l'UI instantanée. La base reste la source de vérité au prochain chargement.

Qualité / transversal

Q : Comment gérez-vous les erreurs ? R : Deux niveaux. (1) Côté **handler** : `try/catch` autour de l'appel Prisma, log `console.error('[canal]', err)` puis `throw` pour propager au front. (2) Côté **composant** : `try/catch` sur l'appel service ; cas métier (FK RESTRICT en Paramètres) affiché à l'utilisateur.

Q : Pourquoi `dev.db` est-il versionné alors que `.gitignore` ignore `*.db` ? R : Volontaire : la base **pré-remplie** (via le seed) est livrée pour que l'app tourne immédiatement à l'évaluation. Le client Prisma généré (`/generated/prisma`), lui, est ignoré → d'où l'étape `npx prisma generate` du README.

11. Points faibles assumés et comment les défendre

L'examineur valorise l'honnêteté lucide. Voici les angles morts du code et la **bonne** façon d'en parler. Mon README contient déjà une section « Remise en question » qui en couvre une partie.

1. **tsconfig.json** **racine pas en strict**. Seul `noImplicitAny: true` y est activé ; en revanche `renderer/tsconfig.json` est **pleinement strict** (`strict` , `strictTemplates` ...). Défense : « *le gros du code applicatif — Angular — est strict ; le tsconfig du main/preload pourrait l'être aussi en une ligne, c'est un oubli de configuration, pas un problème de typage : mes handlers et DTOs sont typés explicitement.* » Correctif : ajouter `"strict": true` au tsconfig racine.
1. **plante.handlers.ts** **n'a pas de try/catch** alors que tous les autres handlers en ont. Défense : « *incohérence assumée ; les 5 autres fichiers de handlers respectent la consigne. C'est un correctif d'une minute : entourer chaque appel.* »
1. **Wikipedia appelé directement depuis le renderer** (`wikipedia.service.ts` , `fetch`) au lieu de passer par un handler IPC. C'est une entorse à « le renderer ne parle qu'au preload », et techniquement un accès réseau externe. Défense : « *c'est de l'enrichissement de saisie optionnel, aucune donnée n'est stockée dans le cloud ; la persistance reste 100 % SQLite local. Idéalement je déplacerais l'appel dans un handler `plantes:scrapeWikipedia` côté main.* » (Le type `plantes:scrapeWikipedia` est d'ailleurs **déclaré** dans `electron.d.ts:16` mais jamais implémenté — vestige du plan initial.)
1. **StatutCulture** **est une table de référence** alors que ses valeurs sont fixes : ça aurait pu être un **enum** (comme `Exposition`). Défense déjà écrite dans le README : table = une jointure + un handler + une section Paramètres « pour rien ». Je sais faire le lien : enum = `TEXT` validé ; table = FK + `RESTRICT` . Contre-argument valable : la table permet à l'utilisateur d'**ajouter** ses propres statuts sans migration.
1. **Pas de contrainte @unique** sur **Parcelle.nom** : deux parcelles peuvent porter le même nom. Correctif : ajouter `@unique` .
1. **app.html** **est du code mort** : c'est le template par défaut d'Angular (logo, « Hello, {{title()}} »), mais `App` utilise un **template inline** (`app.ts:13-18`). Le fichier n'est jamais chargé. À supprimer.
1. **app.spec.ts** **échouerait** : test par défaut qui attend un `h1` « Hello, app » absent de mon `App` . Les tests sont en **bonus** et non lancés par `npm run start` , donc sans impact sur la recevabilité ; à nettoyer ou réécrire si je veux le bonus « tests unitaires ».
1. **ParametresComponent** **utilise ChangeDetectorRef** + mutation d'objets ordinaires, au lieu de signals comme partout ailleurs. Défense : « *ça marche, mais c'est le composant le moins cohérent avec le reste ; je le migrerais en signals pour l'uniformité.* »
1. **Dashboard parStatut** **code en dur les libellés** (`'Planifiée'` , `'En cours'` ...). Si l'utilisateur renomme un statut en Paramètres, le regroupement du dashboard ne le voit plus. Couplage au seed à connaître.

Astuce d'oral : si on vous pointe un de ces défauts, **ne vous braquez pas** — reconnaissez-le, expliquez l'impact réel (souvent faible), donnez le correctif. C'est exactement ce que vaut un développeur.

12. Annexe

12.1 Glossaire express

- **IPC** (*Inter-Process Communication*) : échange de messages entre le renderer et le main (`invoke` / `handle`).
- **contextBridge** / **contextIsolation** : mécanisme Electron qui expose une API du preload au renderer tout en isolant les contextes JS.

- **ORM** : *Object-Relational Mapping* — Prisma traduit mes objets/appels en SQL.
- **DTO** : objet de transfert décrivant la forme exacte des données échangées.
- **Driver adapter** (Prisma 7) : `@prisma/adapters-better-sqlite3`, le pilote concret vers SQLite.
- **Migration** : script SQL versionné décrivant un changement de schéma.
- **Cascade / Restrict / SetNull** : actions `ON DELETE` propagées par SQLite.
- **Signal / Computed / Effect** : primitives de réactivité d'Angular.
- **connectOrCreate** : « relie si existe, sinon crée » (utilisé pour les tags).

12.2 Commandes utiles (les connaître)

Commande	Effet
<code>npm run start</code>	Lance Angular (:4200) + Electron
<code>npm install</code> puis <code>npm install --prefix renderer</code>	Dépendances racine + renderer
<code>npx prisma generate</code>	Régénère le client dans <code>generated/prisma</code>
<code>npx prisma migrate deploy</code>	Applique les migrations (reconstruit la base)
<code>npx prisma db seed</code>	Peuple la base (<code>prisma/seed.ts</code>)
<code>npx @electron/rebuild -f -w better-sqlite3</code>	Recompile le module natif pour Electron
<code>npx prisma studio</code>	Explorateur visuel de la base

12.3 Plan de démonstration suggéré (le jour J)

1. Lancer l'app, montrer le **Dashboard** (parler du `count()` = agrégat SQL).
2. Aller dans **Jardin** : créer une parcelle, faire un **drag & drop** (→ `UPDATE posX/posY`), créer une **culture avec 2 tags** (→ transaction + N:M `connectOrCreate`).
3. Montrer une **récolte**, puis **supprimer la parcelle** → expliquer la **cascade**.
4. **Paramètres** : tenter de supprimer un référentiel utilisé → message d'erreur (**RESTRICT** attrapé).
5. **Plantes** : import **Wikipedia** → `patchValue` du **formulaire réactif** → sauvegarde.
6. Ouvrir `prisma/schema/` et un fichier de **migration** pour montrer le SQL réel (FK, `ON DELETE`).

Gardez ce document ouvert pendant la préparation, pas pendant l'oral. Le but est de pouvoir **raconter** chaque trajet sans le lire.

13. Défense des points faibles — réponses d'oral prêtes

Cette section complète le §11. Le §11 **liste** les faiblesses et leur logique ; ici, ce sont les **phrases à dire à voix haute** quand l'examineur attaque. Règle d'or en 4 temps : « **Oui, c'est exact** » → **la cause** → **l'impact réel** → **le correctif**. Ne jamais nier, ne jamais bluffer. Un examinateur qui voit que vous connaissez le défaut mieux que lui vous donne la note de compréhension. Le réflexe gagnant : ramener chaque critique à du concret que vous maîtrisez — **le SQL équivalent, la frontière de processus, ou le compromis assumé**.

1. « Il manque le `try/catch` dans `plante.handlers.ts` »

C'est votre meilleur coup : la défense **prouve** votre maîtrise de l'IPC.

Réponse :

« Oui, c'est une incohérence : mes 5 autres fichiers de handlers ont le `try/catch`, pas celui-là. Mais ça ne crée pas de trou dans la gestion d'erreur : quand un `ipcMain.handle` asynchrone rejette, Electron **séréalise automatiquement l'erreur et la repropage au `ipcRenderer.invoke()`** du renderer. Or tous mes composants entourent l'appel service d'un `try/catch` — l'erreur Prisma est donc bien attrapée, côté UI. Le `try/catch` du handler, lui, sert surtout au **log serveur** (`console.error('[canal]', err)`) ; c'est ce log que j'ai oublié ici, pas la gestion. Je l'ajouterais pour la cohérence. »

S'il insiste (« mais la consigne l'exige ») : « Vous avez raison sur la consigne, je le concède. Je distingue simplement le respect formel — que je rate sur ce fichier — du risque fonctionnel, qui est nul puisque l'erreur remonte jusqu'au composant. »

2. « Ton TypeScript n'est pas en `strict` »

Réponse :

« Au niveau du `tsconfig.json` racine, je n'active que `noImplicitAny`, c'est exact. En revanche le renderer Angular — le gros du code — est en `strict` complet, avec même `strictTemplates`. Ce qui me manque côté main, c'est surtout `strictNullChecks`. En pratique mon code main/preload est typé explicitement via les DTOs de `shared/ipc`, donc il n'y a pas d' `any` qui traîne. C'est une ligne à ajouter : `"strict": true`. »

S'il ouvre `db.service.ts` et voit le `as any` : « Ça, c'est un `any` **explicite et assumé** : le typage de l'adaptateur `better-sqlite3`, encore en preview dans Prisma 7, ne s'aligne pas parfaitement avec le constructeur du `PrismaClient`. Je caste volontairement pour passer l'adaptateur ; c'est une limite connue de Prisma, pas un relâchement de mon typage. »

3. « Wikipedia, tu l'appelles directement depuis le renderer »

Ici vous **retournez la règle** en montrant que vous savez *pourquoi* elle existe.

Réponse :

« Oui, et je l'ai même écrit dans mon README. Mais creusons la règle : "le renderer ne parle qu'au preload" existe pour une raison de **sécurité** — empêcher le renderer d'accéder aux ressources système (disque, base). Un `fetch` HTTP vers une API publique, c'est exactement ce que le Chromium du renderer fait nativement, en sandbox : ça ne franchit **aucune** barrière de sécurité, contrairement à un accès fichier. Le risque réel est donc nul. Et sur "pas de cloud" : la consigne vise la **persistance** — toutes mes données sont en SQLite local. Wikipedia n'est qu'une aide à la saisie optionnelle ; API coupée = je remplis le formulaire à la main, l'app fonctionne. L'architecture propre serait un handler `plantes:scrapeWikipedia` dans le main — le type est d'ailleurs déjà déclaré dans `electron.d.ts`, c'était mon intention de départ. »

4. « Et si je renomme un statut dans Paramètres ? » (le bug latent du dashboard)

C'est le **seul** qui peut vous piéger en direct. Préparez-le, et si possible **devancez-le**.

Réponse :

« Bonne remarque, et c'est lié à un autre de mes choix. Mon dashboard regroupe les cultures par libellé de statut codé en dur — 'Planifiée', 'En cours' ... Donc si on renomme un statut, le regroupement ne suit pas et la culture disparaît de cette colonne. La cause profonde, c'est d'avoir fait `StatutCulture` en **table éditable** plutôt qu'en `enum` : avec un enum les valeurs seraient garanties, et coder en dur serait légitime. Le correctif propre : itérer sur `this.statuts()` chargés depuis la base et regrouper par `statutId`, pas par libellé. »

Move de pro : si on vous demande « quel point amélioreriez-vous ? », sortez **celui-là vous-même**. Nommer spontanément votre bug le plus subtil = maturité d'ingénieur, et ça oriente la discussion sur un terrain que vous avez préparé.

5. « `app.html` et `app.spec.ts`, c'est le template par défaut »

Réponse courte — ne vous attardez pas, sur-défendre du cosmétique ferait suspect.

Réponse :

« Vestiges du template Angular de départ. `app.html` n'est jamais chargé — `App` utilise un template *inline* dans `app.ts` — et `app.spec` teste le composant par défaut. Zéro impact fonctionnel, mais j'aurais dû faire le ménage. Le type `plantes:scrapeWikipedia` dans `electron.d.ts` est dans le même cas : un reste de mon plan initial. »

6. Modélisation : `@unique` manquant + table vs enum

Réponse (Parcelle.nom) :

« Il manque un `@unique` sur le nom de parcelle, donc deux parcelles peuvent porter le même nom. Sans gravité — chaque parcelle a son `id` et sa position sur la grille — mais plus rigoureux avec la contrainte. En SQL, ça ajouterait un `CREATE UNIQUE INDEX "Parcelle_nom_key" ON "Parcelle"("nom");`. »

Réponse (Statut : table ou enum ?) :

« Choix discutable, que j'assume des deux côtés. Pour la table : l'utilisateur peut **ajouter ses propres statuts sans migration**. Contre : l'enum aurait évité une jointure, un handler et une section Paramètres. La différence SQL : un enum, c'est une colonne `TEXT` validée par Prisma ; une table, c'est une **clé étrangère avec `ON DELETE RESTRICT`**. À refaire, je mettrais Statut en enum et je garderais les tables pour ce qui est vraiment ouvert, comme `TypePlante`. »

7. `ParametresComponent` utilise `ChangeDetectorRef`

Réponse :

« C'est mon seul composant qui n'utilise pas de signals : je stocke mes sections dans des objets ordinaires que je mute, donc Angular ne voit pas le changement tout seul — d'où le `cdr.detectChanges()` manuel. Un signal aurait déclenché le rafraîchissement automatiquement. C'est incohérent avec le reste du projet ; je le réécrirais en signals. »

À retenir : tant que votre phrase contient un « *voici pourquoi* » et un « *voici le correctif* », vous gagnez — même sur un défaut. Une faiblesse expliquée avec lucidité vaut mieux qu'une fonctionnalité que vous ne savez pas

justifier.

14. Questions réellement posées à l'oral — réponses + exemples

Questions effectivement tombées. Pour chacune : une **définition claire**, un **exemple tiré de GardenOS**, et le **SQL** quand c'est pertinent. C'est la section à réviser en priorité.

A. SQL pur

Q1 — Quelle est la différence entre une clé primaire (PK) et une clé étrangère (FK) ?

- **PK (clé primaire)** : identifie **de façon unique** chaque ligne d'une table. Une seule par table, toujours **NOT NULL** + **UNIQUE**. Chez moi : `id Int @id @default(autoincrement())` → en SQL `"id" INTEGER PRIMARY KEY AUTOINCREMENT`.
- **FK (clé étrangère)** : une colonne qui **référence la PK d'une autre table**. Elle matérialise le lien et garantit l'**intégrité référentielle** (impossible de pointer vers un id inexistant). Elle peut être **NULL** (si la relation est optionnelle) et peut **se répéter** (plusieurs lignes vers le même parent).

Exemple GardenOS : dans `Culture`, `id` est la **PK** ; `parcelleId` est une **FK** vers `Parcelle.id`.

```
CREATE TABLE "Culture" (  
  "id"          INTEGER PRIMARY KEY AUTOINCREMENT,  -- PK  
  "parcelleId"  INTEGER NOT NULL,                  -- FK  
  CONSTRAINT "Culture_parcelleId_fkey"  
    FOREIGN KEY ("parcelleId") REFERENCES "Parcelle"("id") ON DELETE CASCADE  
);
```

À savoir : une PK peut être **composite** — `CultureTag` a `@@id([cultureId, tagId])` → `PRIMARY KEY ("cultureId", "tagId")`. Là, deux colonnes qui sont **chacune une FK** forment ensemble la PK.

Q2 — À quoi sert **ON DELETE CASCADE** ?

Quand on supprime la ligne **parent**, le SGBD supprime **automatiquement** les lignes **enfants** qui la référencent. Ça évite les lignes orphelines.

Exemple GardenOS : je supprime une `Parcelle` → SQLite supprime ses `Culture`, ce qui cascade vers leurs `Recolte`, `Journal` et liens `CultureTag`. **Une seule instruction**, tout le sous-arbre part proprement.

```
DELETE FROM "Parcelle" WHERE "id" = 1;  
-- déclenche en chaîne (grâce aux ON DELETE CASCADE) :  
-- DELETE des Culture de la parcelle → DELETE de leurs Recolte/Journal/CultureTag
```

Les 3 comportements que j'utilise (à opposer) :

- **CASCADE** : supprime les enfants (`Culture`→`Recolte`/`Journal`).
- **RESTRICT** (défaut FK obligatoire) : **bloque** la suppression si des enfants existent (supprimer un `StatutCulture` utilisé échoue).
- **SET NULL** (défaut FK optionnelle) : met la FK à **NULL** (supprimer un `TypeSol` → la parcelle a `typeSolId = NULL`).

Sur SQLite, ça ne marche que si `PRAGMA foreign_keys = ON`. C'est le moteur SQLite qui propage, pas Prisma.

Q3 — Qu'est-ce qu'un JOIN ? Expliquer la différence entre tous les types.

Un **JOIN** combine les lignes de deux tables sur une condition (souvent `FK = PK`).

Type	Ce qu'il garde
INNER JOIN	Seulement les lignes qui ont une correspondance des deux côtés
LEFT (OUTER) JOIN	Toutes les lignes de gauche + les correspondances à droite (NULL si aucune)
RIGHT (OUTER) JOIN	Toutes celles de droite + correspondances à gauche (NULL sinon)
FULL (OUTER) JOIN	Toutes les lignes des deux côtés (NULL là où ça ne matche pas)
CROSS JOIN	Produit cartésien : chaque ligne de gauche × chaque ligne de droite

Exemple GardenOS (parfait car `StockItem.planteId` est optionnel) :

```
-- INNER JOIN : seuls les articles LIÉS à une plante (ex: "Graines de tomate")
SELECT s."nom", p."nom"
FROM "StockItem" s
INNER JOIN "Plante" p ON p."id" = s."planteId";

-- LEFT JOIN : TOUS les articles ; plante = NULL pour "Arrosoir 10L" et "Compost"
SELECT s."nom", p."nom"
FROM "StockItem" s
LEFT JOIN "Plante" p ON p."id" = s."planteId";
```

Le lien avec mon code : mon handler `stocks:getAll` fait `include: { categorie: true, plante: true }`. `categorie` est obligatoire → comportement type **INNER** ; `plante` est optionnelle → comportement type **LEFT** (l'article reste affiché même sans plante liée).

Détail SQLite : **INNER**, **LEFT** et **CROSS** sont supportés depuis toujours ; **RIGHT** et **FULL OUTER** seulement depuis SQLite 3.39 (2022). Un **RIGHT JOIN** peut toujours se réécrire en **LEFT JOIN** en inversant les tables.

Q4 — Différence entre **WHERE** et **HAVING** ?

- **WHERE** filtre les **lignes** *avant* le regroupement. Il **ne peut pas** utiliser de fonction d'agrégat (**COUNT**, **SUM** ...).
- **HAVING** filtre les **groupes** *après* **GROUP BY**. Il **peut** porter sur un agrégat.

Exemple GardenOS (« combien de cultures par statut, et ne garder que les statuts avec plus d'1 culture ») :

```
SELECT "statutId", COUNT(*) AS nb
FROM "Culture"
WHERE "dateSemisReelle" IS NOT NULL -- filtre les lignes AVANT regroupement
GROUP BY "statutId"
HAVING COUNT(*) > 1; -- filtre les groupes APRÈS agrégation
```

Lien avec mon code : mon dashboard fait ce regroupement « cultures par statut » en **JavaScript** (un `computed` `parStatut`). En SQL pur, ce serait exactement `GROUP BY "statutId"` (+ `HAVING` pour filtrer). Et mon compteur du dashboard, `db.plante.count()`, c'est `SELECT COUNT(*) FROM "Plante"` — l'agrégat le plus simple.

B. ORM / Prisma

Q5 — Qu'est-ce qu'un ORM ?

****ORM = Object-Relational Mapping. Une couche qui fait correspondre les tables de la base à des objets/modèles dans le code. J'écris des appels typés (`db.plante.findMany(...)`) et l'ORM génère le SQL**** à ma place, mappe les résultats en objets, et gère les migrations.

- **Avantages** : sécurité de typage (TypeScript), productivité, pas d'injection SQL par défaut (requêtes paramétrées), portable entre SGBD.
- **Inconvénients** : l'abstraction peut **masquer** des requêtes coûteuses (problème N+1) ; moins de contrôle fin que du SQL écrit à la main.

Mon ORM ici, c'est **Prisma**, branché sur SQLite via l'adaptateur `@prisma/adapters-better-sqlite3`.

Q6 — Différence entre `prisma migrate` et `prisma generate` ?

- `prisma migrate` agit sur la **BASE** : il crée et applique des fichiers **SQL de migration** (du DDL : `CREATE TABLE`, `ALTER TABLE` ...) pour faire évoluer le schéma. Mes 7 migrations sont dans `prisma/migrations/`.
- `prisma generate` agit sur le **CODE** : il (re)génère le **client TypeScript** dans `generated/prisma` à partir du schéma. **Aucune** modification de la base — juste l'API typée que j'importe (`PrismaClient`).

Mnémonique : *migrate = la base · generate = le code*. C'est pour ça que le README demande `prisma generate` après l'install (le client n'est pas versionné, il est dans `.gitignore`).

Q7 — Comment déclare-t-on une relation 1-N dans Prisma ?

Le côté « **N** » porte la **clé étrangère** + le `@relation` ; le côté « **1** » a juste un **tableau** (champ virtuel, pas une colonne).

Exemple exact GardenOS (`Parcelle` 1 — N `Culture`) :

```
model Parcelle {
  id      Int      @id @default(autoincrement())
  cultures Culture[] // côté "1" : tableau virtuel
}
model Culture {
  parcelleId Int // la FK (côté "N")
  parcelle   Parcelle @relation(fields: [parcelleId], references: [id], onDelete: Cascade)
}
```

`fields: [parcelleId]` = la colonne FK locale ; `references: [id]` = la PK ciblée. En SQL ça devient le **FOREIGN KEY** ("parcelleId") REFERENCES "Parcelle"("id") .

Q8 — À quoi sert `seed.ts` ?

À **peupler la base** avec un jeu de données initial / de test, lancé via `npx prisma db seed` . Utile pour : démarrer sur un état connu, faire une démo, tester.

Mon `seed.ts` : il **nettoie** d'abord les tables dans l'ordre des dépendances (`deleteMany`), puis crée les référentiels (types, statuts, catégories), 5 plantes, 3 parcelles, des tags, des cultures (dont une avec tags via `connect`), des récoltes, des entrées de journal et des articles de stock. C'est ce qui garnit `dev.db` , que je livre déjà rempli.

C. Electron

Q9 — Quels sont les 2 processus principaux d'Electron ? Pourquoi ne pas appeler Prisma depuis le `renderer` ?

- **Main** (processus Node.js) : accès complet à l'OS, fenêtres, fichiers, modules natifs. C'est lui qui détient Prisma.
- **Renderer** (processus Chromium) : affiche l'UI (mon app Angular), **sandboxé** comme un onglet de navigateur.

Pourquoi pas Prisma dans le `renderer` : (1) **technique** — le `renderer` n'a pas les API Node (`fs` , etc.) ni le module natif `better-sqlite3` ; (2) **sécurité** — avec `contextIsolation: true` / `nodeIntegration: false` , le `renderer` ne peut pas toucher au disque. Donc tout l'accès données vit dans le **main**, et le `renderer` le sollicite par **IPC**.

Q10 — C'est quoi un *preload* ? (et : « le preload autorise le frontend à communiquer avec le backend »)

Un ****script** qui s'exécute *avant* le chargement de la page **du renderer, dans un contexte privilégié qui voit à la fois un sous-ensemble de Node et le futur `window`**. Son rôle : exposer une API contrôlée **du backend (main) vers le frontend (renderer)**, via **`contextBridge`**. C'est le pont — et la seule** porte d'entrée du renderer vers le main.

Mon **`preload.ts`** déclare une liste blanche de canaux :

```
contextBridge.exposeInMainWorld('electronAPI', {
  'stocks:getAll': () => ipcRenderer.invoke('stocks:getAll'),
  'stocks:delete': (data) => ipcRenderer.invoke('stocks:delete', data),
  // ... un canal par opération
});
```

Côté renderer, mes services appellent **`window.electronAPI['stocks:delete']({ id })`**.

Q11 — C'est quoi le *context bridge* ?

`contextBridge.exposeInMainWorld(nom, objet)` est l'API Electron qui **injecte de façon sûre** un objet du preload dans le **`window`** du renderer, **malgré `contextIsolation: true`**. Cette isolation sépare les contextes JavaScript (preload vs page) pour qu'une faille dans la page ne puisse pas atteindre les privilèges du preload. Sans le **`contextBridge`**, le renderer ne « verrait » rien de ce que le preload définit.

Q12 — Pourquoi SQLite ?

- **Léger** : une bibliothèque C embarquée, pas de serveur de base à installer/lancer (*server/less*).
- **Portable** : toute la base tient dans **un seul fichier** (**`dev.db`**) qu'on peut copier, versionner, livrer.
- **Zéro configuration**, parfait pour une **application de bureau mono-utilisateur**.
- **Imposé par la consigne** (**`provider = "sqlite"`**).

Limite à connaître : SQLite n'est pas taillé pour de **fortes écritures concurrentes multi-utilisateurs** (il verrouille au niveau du fichier). Pour une app desktop locale, c'est idéal.

Q13 — Que se passe-t-il si on active **`nodeIntegration`** ? (réponse en un mot : **`true`**)

Si **`nodeIntegration: true`**, le **renderer obtient l'accès complet à Node.js** (**`require`**, **`fs`**, **`process`** ...).

Danger majeur : une simple faille **XSS** dans la page donnerait à un attaquant un accès complet à la machine (lecture/écriture de fichiers, exécution...). C'est pourquoi je le laisse à **`false`** + **`contextIsolation: true`** : le renderer ne peut faire **que** ce que mon preload expose.

Q14 — Revoir **`ipcMain.handle`** et le fameux **`_e`**

`ipcMain.handle('canal', listener)` enregistre, côté **main**, le **gestionnaire** d'un canal appelé par **`ipcRenderer.invoke('canal', data)`** côté renderer. Le **`listener`** reçoit (**`event, ...args`**) : le **1er paramètre est l'événement IPC** (**`IpcMainInvokeEvent`** : **`sender`**, etc.), souvent **inutile**.

Le **`_e`** / **`_event`** : quand je n'utilise pas ce 1er paramètre, je le préfixe d'un **underscore** pour signaler « **volontairement ignoré** » (convention, + évite l'avertissement « unused parameter »).

```
// refs.handlers.ts – l'événement est ignoré (_e), je n'utilise que le DTO
ipcMain.handle('typePlantes:create', async (_e, dto) => db.typePlante.create({ data: dto }));

// plante.handlers.ts – j'ignore l'event, je déstructure l'id de l'argument
ipcMain.handle('plantes:delete', async (_event, { id }) => { await db.plante.delete({ where: { id }
}); });
```

Q15 — `main.ts` / `before-quit` : couper la connexion à la base (`prisma.$disconnect`)

À la fermeture de l'app, il faut **fermer proprement** la connexion Prisma/SQLite pour libérer le fichier et éviter un verrou résiduel. **C'est déjà dans mon projet** :

```
// main.ts
app.on('before-quit', async () => { await disconnectDb(); });

// db.service.ts
export async function disconnectDb() {
  if (_prisma) { await _prisma.$disconnect(); _prisma = null; }
}
```

Je peux donc l'annoncer : « la connexion est fermée sur `before-quit` via `$disconnect()` ». Mon client Prisma est par ailleurs un **singleton** (`getDb()` ne crée l'instance qu'une fois).

Q16 — À quoi sert le `.env` ?

À stocker les **variables d'environnement** (configuration, URLs, secrets) **en dehors du code**, pour séparer config et code et **ne pas versionner** les valeurs sensibles (il est dans `.gitignore`).

Mon `.env` : `DATABASE_URL="file:./dev.db"` . Il est chargé par `dotenv` (`import 'dotenv/config'` dans `seed.ts` et `prisma.config.ts`), et Prisma lit `DATABASE_URL` pour savoir **où se trouve la base**.

D. Angular

Q17 — Pourquoi un `signal` et pas un `computed` ? Que fait le signal ?

- `signal` = une **source d'état modifiable**. Je le lis en l'appelant (`recherche()`) et je le change avec `.set(...)` ou `.update(...)` . À utiliser quand la valeur est **posée directement** (saisie utilisateur, réponse d'un appel, ouverture d'une modale).
- `computed` = une valeur **dérivée, en lecture seule, recalculée automatiquement** quand les signals qu'elle lit changent (et **mise en cache** sinon). On **ne peut pas** lui faire `.set()` .

Règle : si je dois *écrire* la valeur → `signal` . Si elle se *déduit* d'autres → `computed` .

Exemple GardenOS :

```
recherche      = signal('');           // SET par l'utilisateur (input)
filtreCategorie = signal<number | null>(null);
stocks         = signal<StockItem[]>([]); // SET par la réponse IPC

// dérivé → computed (jamais .set, recalculé seul)
stocksFiltres = computed(() =>
  this.stocks().filter(s =>
    s.nom.toLowerCase().includes(this.recherche().toLowerCase())
  )
);
```

Si je faisais `stocksFiltres` en signal, je devrais le **recalculer à la main** à chaque frappe — le `computed` le fait tout seul.

Q18 — Comment exposer les fonctions du backend vers le frontend (renderer) ?

La chaîne complète (4 maillons) :

1. **Main** enregistre un gestionnaire : `ipcMain.handle('stocks:delete', (_e, {id}) => db.stockItem.delete({where:{id}}))` .

2. **Preload** expose un canal via `contextBridge` : `'stocks:delete': (d) => ipcRenderer.invoke('stocks:delete', d)`.
3. **electron.d.ts** type `window.electronAPI` pour le renderer (sinon ce serait `any`).
4. **Render** appelle `window.electronAPI['stocks:delete']({ id })`, encapsulé dans un **service** Angular.

C'est exactement le trajet de mon « flux complet » (§4) : composant → service → preload → handler → Prisma → SQLite, puis retour.

Q19 — Qu'est-ce qu'une route Angular ?

Une **route** associe un **chemin d'URL** à un **composant**, affiché dans le `<router-outlet>`. Je déclare les routes dans `app.routes.ts`, je les active avec `provideRouter(routes)` (dans `app.config.ts`), et je navigue avec `routerLink`.

Mon `app.routes.ts` (7 routes) :

```
export const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }, // redirection
  { path: 'dashboard', component: DashboardComponent },
  { path: 'jardin', component: JardinComponent },
  // ... plantes, stock, journal, parametres
];
```

La barre de nav utilise `routerLink="/jardin"` + `routerLinkActive="active"` pour surligner l'onglet courant.

E. Consigne transversale de l'examineur

« Chaque requête Prisma doit être comprise et pouvoir être refaite en SQL. »

C'est le cœur de l'épreuve. Tout est outillé dans ce dossier : la **table de correspondance** Prisma → SQL (§6) et les **8 flux « bouton → SQL »** (§7). Le réflexe à avoir devant n'importe quel appel :

Méthode Prisma	Mot-clé SQL
<code>findMany</code> / <code>findUnique</code>	<code>SELECT ... (WHERE ...) (ORDER BY ...)</code>
<code>create</code>	<code>INSERT INTO ... VALUES ... RETURNING *</code>
<code>update</code>	<code>UPDATE ... SET ... WHERE ...</code>
<code>delete</code> / <code>deleteMany</code>	<code>DELETE FROM ... WHERE ...</code>
<code>count</code>	<code>SELECT COUNT(*) ...</code>
<code>include</code>	<code>JOIN</code> sur la clé étrangère
<code>connectOrCreate</code>	<code>SELECT</code> (existe ?) puis <code>INSERT</code> sinon
écriture imbriquée	le tout dans une transaction <code>BEGIN ... COMMIT</code>

Entraînez-vous à prendre **un handler au hasard** et à dire son SQL à voix haute.

Bonne défense. Le code est solide ; l'enjeu est de prouver que vous le comprenez ligne à ligne.